

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»

Университетский технологический колледж

наименование подразделения

Кафедра Информационные системы в экономике

наименование кафедры

Направление Информационные системы и программирование

Отчёт защищён с оценкой _____

_____ К.В Воробьев

(подпись руководителя от вуза) (инициалы, фамилия)

“ _____ ” _____ 2025г.

ОТЧЕТ

по лабораторной работе №4

Некоторые конструкции

тема лабораторной работы

по дисциплине Объектно-ориентированное программирование

ЛР 09.02.07.28.001 ПЗ

обозначение документа

Студент группы _____ 1ИСП-22

_____ М.С. Заковряшин

инициалы, фамилия

Руководитель работы _____ преподаватель

должность, ученое звание

_____ К.В. Воробьев

инициалы, фамилия

Барнаул 2025

Задание 1: Обработка исключений

- Используйте try-except для обработки исключений.

```
def on_square_click(self, row, col):
    try:
        piece = self.board.get_field()[row][col]
        if self.selected_piece is None:
            if piece is not None:
                self.selected_piece = piece
                self.selected_position = (row, col)
                print(f"Выбрано: {piece.get_color()} позиция {row}, {col}")
            else:
                if piece is None:
                    self.board.get_field()[row][col] = self.selected_piece
                    self.board.get_field()[self.selected_position[0]][self.selected_position[1]] = None
                    self.update_square(row, col)
                    self.update_square(self.selected_position[0], self.selected_position[1])
                    self.selected_piece = None
                    self.selected_position = None
        except IndexError:
            print("НЕВОЗМОЖНЫЙ ХОД : НЕВЕРНАЯ ПОЗИЦИЯ")
            raise # Повторно выбрасываем исключение, если это необходимо
```

Рисунок 1 - использование конструкции трай екпект

- Добавьте finally для выполнения обязательного кода.

```
except Exception as e:...
finally:
    print(f"Обработка клика по квадрату завершена: {row}, {col}.")
```

Рисунок 2 - использование finnaly

- Используйте raise для генерации собственных исключений.

```
except IndexError:
    raise OutOfBoundsError("Выход за пределы доски.")
except ChessError as e:
```

Рисунок 3 - использование raise для генерации своего исключения

- Создайте иерархию пользовательских исключений (например, BaseException -> CustomError -> SpecificError). Обработайте каждое исключение по-разному в зависимости от его типа.

```

class ChessError(Exception):
    """Основная ошибка"""
    pass

class InvalidMoveError(ChessError):
    """Исключение, возникающее при попытке осуществить недопустимый ход."""

    def __init__(self, message="Недопустимый ход"):
        self.message = message
        super().__init__(self.message)

class PieceNotSelectedError(ChessError):
    """Исключение, возникающее, когда игрок пытается переместить фигуру, не выбрав ее."""

    def __init__(self, message="Фигура не выбрана"):
        self.message = message
        super().__init__(self.message)

class OutOfBoundsError(ChessError):
    """Исключение, возникающее при выходе за пределы доски."""

    def __init__(self, message="Выход за пределы доски"):
        self.message = message
        super().__init__(self.message)

```

Рисунок 4 - мои исключения

Задание 2: Работа с массивами объектов

- Создайте класс и продемонстрируйте работу с одномерным и двумерным списками его объектов.

```

def __init__(self):
    self.color = 'WHITE'
    self.field = []
    for row in range(8):
        self.field.append([None] * 8)
    self.field[0] = [
        Rook('WHITE'), Hourse('WHITE'), Bishop('WHITE'), Queen('WHITE'),
        King('WHITE'), Bishop('WHITE'), Hourse('WHITE'), Rook('WHITE')
    ]
    self.field[1] = [
        Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'),
        Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE')
    ]
    self.field[6] = [
        Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'),
        Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK')
    ]
    self.field[7] = [
        Rook('BLACK'), Hourse('BLACK'), Bishop('BLACK'), Queen('BLACK'),
        King('BLACK'), Bishop('BLACK'), Hourse('BLACK'), Rook('BLACK')
    ]

```

Рисунок 5 - массив объектов класса

В классе доска создается массив объектов классов фигур

- Реализуйте метод, который находит объект с максимальным значением определенного атрибута в двумерном списке. Убедитесь, что метод корректно обрабатывает пустые списки.

```

def create_random_matrix():
    matrix = []
    for i in range(10):
        matrix_elem = []
        for j in range(random.randint(a: 1, b: 10)):
            matrix_elem.append(random.randint(a: 1, b: 2450))
        else:
            matrix.append(matrix_elem)
    return matrix

```

Рисунок 6 - создание матрицы

```

matrix = create_random_matrix()

def serch_elem_matrix(matrix):
     max_elem = 0
    if len(matrix) >= 1:
        for matrix_string in matrix:
            for elem in matrix_string:
                if elem > max_elem:
                    max_elem = elem
            else:
                return max_elem
    else:
        return 0

maximum_matrix_elem = serch_elem_matrix(matrix)
print(maximum_matrix_elem)

```

Рисунок 7 - поиск максимального элемента в матрице

Задание 3: Наследование

- Создайте базовый и производный классы.
- Переопределите метод базового класса в производном.
- Продемонстрируйте вызов метода базового класса из производного.
- Добавьте в производный класс метод, который использует как переопределенный метод, так и метод базового класса, но в разной последовательности в зависимости от условия.

Все это есть в структуре абстрактного класса фигура, методы его переопределены в конкретных фигурах

Задание 4: Защищенные атрибуты

- Используйте `_` для обозначения защищенных атрибутов.

- Продемонстрируйте доступ к ним в производном классе.

У меня в коде просто нет возможности сделать защищенные атрибуты.

Задание 5: Конструкторы и наследование

- Создайте конструктор в производном классе, вызывающий конструктор базового через `super()`.

```
class InvalidMoveError(ChessError):
    """Исключение, возникающее при попытке осуществить недопустимый ход."""
    def __init__(self, message="Недопустимый ход"):
        self.message = message
        super().__init__(self.message)

class PieceNotSelectedError(ChessError):
    """Исключение, возникающее, когда игрок пытается переместить фигуру, не выбрав ее."""
    def __init__(self, message="Фигура не выбрана"):
        self.message = message
        super().__init__(self.message)
```

Рисунок 8 - создание конструктора с использованием `super`

Работа с абстрактным классом, была продемонстрирована в лекции ранее

Задание 6: Строковое представление

- Переопределите `__str__` для вывода информации об объекте.
- Реализуйте метод `__repr__` так, чтобы он возвращал строку, которую можно использовать для повторного создания объекта (например, с помощью `eval`).

```

class Rook(Figure):
    def __init__(self, color):
        super().__init__(color)

    def can_move(self, board, row, column, row1, column1):
        return row == row1 or column == column1

    def can_attack(self, board, row, column, row1, column1):
        return self.can_move(board, row, column, row1, column1)

    def get_color(self):
        return super().get_color()

    def __str__(self):
        return f"Rook(color='{self.color}')"

    def __repr__(self):
        return f"Rook(color='{self.color}')"

class Pawn(Figure):

```

ook

Рисунок 9 - перегрузка операторов

Оператор str теперь вернет цвет ладьи.

```

D:\УШАКОВ\chess_lr2\.venv\Scripts\python.exe "D:\00П\Лр 3\chess_zakovryashin_lr2_v0_3.py"
Rook(color='black')
Rook(color='black')
Rook(color='black')

```

Рисунок 10 - демонстрация работы программы

```
root = tk.Tk()
app = ChessApp(root)
root.mainloop()
```

```
rook = Rook('Black')
print(rook)
print(repr(rook))
```

```
new_rook = eval(repr(rook))
print(new_rook)
```

Рисунок 11 - применение функций eval и repr

https://github.com/LordFarqa/zakovryashin_1_isp_23_0/tree/object-oriented-programming